SCHOLAR AI

***AI-Assisted Scholarship Discovery, Live Web Extraction & Deterministic*

***Version:** 2.1.0*

An intelligent engineering system combining real-time Google Serper search,

***Author & Architect:** Full-Stack AI Engineering Team*

***Repository:** [Scholar AI](file:///d:/Scholar%20AI)*

***Backend Framework:** FastAPI (Python 3.11+)*

***Frontend Framework:** React 18 + Vite (SPA)*

***Database:** MongoDB Atlas (PyMongo)*

***Authentication:** Dual-Hasher Argon2id / Legacy Bcrypt + JWT (HS256)*

Table of Contents

1. [Chapter 1: Project Introduction](#chapter-1-project-introduction)
2. [Chapter 2: Technology Stack](#chapter-2-technology-stack)
3. [Chapter 3: System Architecture](#chapter-3-system-architecture)
4. [Chapter 4: Repository and Folder
5. [Chapter 5: Frontend Implementation](#chapter-5-frontend-implementation)
6. [Chapter 6: Backend Implementation](#chapter-6-backend-implementation)
7. [Chapter 7: API Documentation](#chapter-7-api-documentation)
8. [Chapter 8: Database Documentation](#chapter-8-database-documentation)
9. [Chapter 9: Authentication and Security](#chapter-9-authentication-and-security)
10. [Chapter 10: Scholarship Discovery
11. [Chapter 11: Eligibility Rule Engine](#chapter-11-eligibility-rule-engine)
12. [Chapter 12: Scholarship Matching and
13. [Chapter 13: Saved Scholarships Feature](#chapter-13-saved-scholarships-feature)
14. [Chapter 14: Application Tracker Feature](#chapter-14-application-tracker-feature)
15. [Chapter 15: Complete End-to-End
16. [Chapter 16: Installation and Local
17. [Chapter 17: Environment Variables
18. [Chapter 18: Testing and Quality
19. [Chapter 19: Deployment Guide](#chapter-19-deployment-guide)
20. [Chapter 20: Troubleshooting Guide](#chapter-20-troubleshooting-guide)

SYSTEM SPECIFICATION & ARCHITECTURAL AUDIT

Platform Name:	Scholar AI (Production Milestone 2.1.0)
Target Demographics:	Indian College & University Students (B.Tech, MBBS, B.Sc, B.Com, etc.)
Frontend Stack:	React 18 Single Page Application, Vite 5, React Router v6, Pure CSS
UI Design System:	Modern Clean Dark SaaS (Linear & Stripe Design Tokens)
Backend Framework:	FastAPI 0.115+, Python 3.11+, Pydantic V2, Uvicorn ASGI Server
Database Engine:	MongoDB Atlas (Cloud M0 Replica Set), PyMongo 4.6+, TLS CA Handshake
Authentication:	Argon2id Multi-Hasher, Legacy Bcrypt Auto-Upgrade, 7-Day JWT (HS256)
Discovery Pipeline:	Serper Google Search API, BS4 Scraper, Rupee Heuristic with Rs.500 Floor
Rule Evaluation:	Deterministic 9-Rule Engine (Income, Category, Domicile, Cutoff, Synonyms)
Test Coverage:	53 Passing Pytest Unit & Integration Tests (100% Core Regression Pass)
Generated Date:	November 2026

expired scholarships with obsolete deadlines or host ad-heavy clickbait redirects

21. [Chapter 21: Technical Decisions and

22. [Chapter 22: Limitations and Future

23. [Chapter 23: Complete Technical Interview

document submission, application deadlines, status tracking (Applied, Under Review,

24. [Chapter 24: Technical Glossary](#chapter-24-technical-glossary)

(normalizing Lakh/Crore expressions), and detect deadlines.

Chapter 1: Project Introduction

1.1 Project Overview

Scholar AI is a full-stack, intelligent web platform designed to democratize higher

links without explaining *why* the user is or is not eligible. A student rejected for

By combining **live web search (Serper Google API), Human-readable explanations for every rule evaluated.

utilizing the student's exact demographic, geographic, and academic attributes.

1.2 Problem Statement

In India, hundreds of crores in scholarship funds lapse or go unclaimed each academic year

1. **Severe Information Asymmetry:** Schemes are scattered across Central Government

   parse HTML, strip commercial clickbait domains, extract rupee award values

   verified opportunities first.

2. **Ambiguous & Complex Eligibility Criteria:** Eligibility notices feature dense legal

   eligibility audit ('ELIGIBLE', 'NEEDS_VERIFICATION', 'INELIGIBLE') accompanied by

3. **Stale Information & Fraudulent Portals:** Many popular educational portals list

4. **Lack of Centralized Tracking:** Once a student discovers a scholarship, managing

   interactive Kanban-style pipeline.

5. **Opaque "Black-Box" Recommendations:** Typical AI or search systems return a list of

   strictly on the ratio of passed criteria over evaluated criteria, prioritizing fully

1.3 Objectives

- **Targeted Discovery:** Generate personalized, context-aware Google Search queries

  scholarships to an isolated private profile and track application statuses through an

- **Automated Data Extraction & Sanitization:** Automatically fetch live web results,

- **Transparent Deterministic Eligibility:** Execute a strict, transparent 9-point

- **Accurate Mathematical Matching:** Compute a normalized match percentage based

- **End-to-End Application Lifecycle Management:** Empower students to save prospective

1.4 Target Users

behalf (government and CSR portals require direct Aadhaar biometric authentication or

- **Undergraduate & Postgraduate Students:** Enrolled in professional (B.Tech, MBBS,
- **Economically Weaker Section (EWS) & Low-Income Families:** Students requiring fee
- **Reserved & Special Category Students:** SC, ST, OBC, EWS, and minority candidates
- **Single Girl Child & Female Scholars:** Female students seeking gender-specific
- **Persons with Disabilities (PwD):** Differently abled scholars seeking specialized

1.5 Real-World Use Cases

SCHOLAR AI TECH STACK

1. **First-Generation Engineering Student:** A 2nd-year B.Tech student from Maharashtra
2. **Medical Student Seeking Corporate CSR Support:** An MBBS student with an Rs.
3. **Application Deadline Management:** A student managing three concurrent applications

1.6 Project Scope & Limitations

- **Current Scope:** Live search across public web sources via Serper API, in-memory +
- **Current Limitations:** The platform does not submit applications on the student's

Chapter 2: Technology Stack

routing, and real-time state synchronization.

providers (AuthContext),

```
================================================================================...
```

universal developer familiarity.

'useCallback', 'useMemo', 'useRef', 'useNavigate', 'useLocation'), custom context

2.1 Frontend Technologies

React 18 (Single Page Application)

- **Role:** Drives the responsive user interface, component hierarchy, client-side
- **Why Chosen:** Declarative UI model, virtual DOM reconciliation, rich ecosystem, and
- **Key Modules Used:** Functional components, hooks ('useState', 'useEffect',

Vite 5

serialization

- **Role:** Next-generation frontend build tool and local development server.
- **Why Chosen:** Native ESM-based development server providing sub-50ms Hot Module validation; automatic JSON schema generation and field validators for data hygiene

- **Key Configuration:** Proxies '/api' routes during development to

React Router v6

dynamically upgraded to Argon2id without interrupting the user.

- **Role:** Declarative client-side routing, route protection guards, URL query

- **Key Implementation:** Handles seamless transitions between '/', '/scholarships', clusters memory and time cost parameters.

Vanilla Modern CSS (Design System: Linear & Stripe SaaS)

- **Role:** Pure CSS custom property (variable) architecture without external heavy document modeling, and native compound indexes. Configured with 'certifi.where()'

- **Design Aesthetic:** Deep space dark slate canvas ('#0B0F19', '#111827',

2.2 Backend Technologies

successful authentication of legacy '$2b$' Bcrypt hashes, the user's password hash is

FastAPI (Python 3.11+)

- **Role:** High-performance, asynchronous REST API server.
- **Why Chosen:** Native Python type hints, automatic OpenAPI/Swagger documentation

- **Architecture:** 12 modular routers connected via an async lifespan lifecycle providing state-of-the-art resistance against GPU/ASIC brute-force attacks via tunable

Pydantic v2

- **Role:** Request and response data validation, strict schema enforcement, and type

- **Why Chosen:** Written in Rust core ('pydantic-core') for blazingly fast

BcryptHasher()))' with an automated fallback to 'bcrypt.checkpw()'. Upon

PyMongo 4.6+ with MongoDB Atlas

- **Role:** Official synchronous Python driver connecting to cloud-hosted MongoDB Atlas

- **Why Chosen:** Schema flexibility accommodates diverse scholarship structures, rich

2.3 Security & Authentication Technologies

Argon2id ('pwdlib') + Legacy Bcrypt Fallback

- **Role:** Industrial-grade password hashing engine.
- **Why Chosen:** Argon2id is the winner of the Password Hashing Competition (PHC),

- **Dual-Hasher Architecture:** Configured via 'PasswordHash((Argon2Hasher(), expiration ('exp'). Extracted securely via FastAPI 'HTTPBearer' dependency.

PyJWT (JSON Web Tokens)

- **Role:** Stateless, cryptographically signed user session tokens.
- **Algorithm:** HMAC with SHA-256 ('HS256').
- **Payload:** User ID ('sub'), email, issuer ('iss: scholar-ai'), and 7-day

2.4 Scraping & Search Services

Serper Google Search API

- **Role:** Fast, reliable Google Search engine API returning structured search

- **Integration:** Asynchronous and synchronous fallback wrappers with dynamic query

BeautifulSoup4 ('bs4') & Requests

- **Role:** Web scraping, DOM traversal, and semantic text extraction.
- **Responsibilities:** Fetches target scholarship pages, removes script/style noise,

Chapter 3: System Architecture

3.1 High-Level Architecture Overview

Scholar AI is structured as a decoupled, multi-tier client-server architecture:

1. **Presentation Layer (Frontend):** React SPA hosted on Vercel, interacting with
2. **API & Orchestration Layer (FastAPI):** Cloud-hosted on Render, managing CORS,
3. **Domain Service Layer:** Pure business logic handling deterministic rule evaluation,
4. **Data Access Layer (Repositories):** Encapsulated MongoDB operations utilizing
5. **Persistence & External Service Layer:** MongoDB Atlas (M0 replica set) and Serper





3.2 Frontend-Backend Communication Flow

```
sequenceDiagram
```

3.3 Authentication & Authorization Flow

```
sequenceDiagram
```

```
Start([User Initiates Discovery]) --> FetchProfile[Fetch Student Profile fro...
```

```
AuthService->>Sec: create_access_token\(user_id, email\)
```

3.4 Scholarship Discovery Workflow

```
flowchart TD
```

3.5 Eligibility Evaluation Workflow

autonumber

```
flowchart TD
```

3.6 Saved Scholarship Workflow

```
sequenceDiagram
```

3.7 Application Tracker Workflow

```
sequenceDiagram
```

Chapter 4: Repository and Folder Structure

The Scholar AI repository adheres to a clean separation of concerns:

```
d:\\Scholar AI\\
```

```
__init__.py
```

```
application_service.py # Application tracking lifecycle business l...
```

```
App.css                # Linear/Stripe SaaS design system & responsiv...
```

CSS system inspired by **Linear** and **Stripe**. The UI emphasizes visual hierarchy, high surfaces (‘#111827', ‘#1F2937'). for primary calls to action, active sidebar links, and match score rings. 0.12)'). 158, 11, 0.12)'). 0.12)'). figures and percentage scores.

Chapter 5: Frontend Implementation

5.1 Architecture & Design System

The frontend is constructed using React 18, bundled with Vite 5, and styled using a custom

Visual Palette & Design Tokens (‘App.css‘)

- **Background Slate:** Deep space dark canvas (‘#0B0F19‘) paired with elevated card

([Layout.jsx](file:///d:/Scholar%20AI/frontend/src/components/Layout.jsx))

- **Brand Accents:** Electric Indigo (‘#6366F1‘) and Violet (‘#8B5CF6‘) gradients
- **Semantic Badges:**
- *Eligible / High Match:* Emerald Green (‘#10B981‘, background ‘rgba(16, 185, 129, trail, user profile pill, and dynamic content viewport.
- *Needs Verification / Medium Match:* Amber Gold (‘#F59E0B‘, background ‘rgba(245,
- *Ineligible / Low Match:* Crimson Rose (‘#EF4444‘, background ‘rgba(239, 68, 68,
- **Typography:** Inter sans-serif typeface paired with monospace numbers for financial

5.2 Key Frontend Components

1. ‘Layout.jsx‘

- **Responsibilities:** Renders the persistent left navigation sidebar, breadcrumb
- **Key Features:**
- Active route highlighting with glow indicators.
- Collapsible mobile navigation drawer with hamburger toggle.
- Safe desktop layout without duplicate bottom navigation bars.

Applications in Progress, and *Total Award Potential*.

- Direct logout trigger tied to ‘AuthContext‘.

2. ‘ScholarshipCard.jsx‘

- **Responsibilities:** Displays individual scholarship summaries across Discovery,
- **Key Features:**
- Radial match percentage badge with dynamic color transitions ($>75\%$ emerald, ([Scholarships.jsx](file:///d:/Scholar%20AI/frontend/src/pages/Scholarships.jsx))
- Parsed award amount highlighted in prominent bold rupee figures (e.g., ‘Rs. 50,000 /
- Deadline countdown pill indicating remaining days with warning colors for urgent
- Optimistic Save/Unsave bookmark button with instant visual response.
- Direct link to official application portal (with ‘rel="noopener noreferrer"‘

3. ‘Icons.jsx‘

Discovery* (real-time Serper scraping & evaluation).

- **Responsibilities:** Self-contained SVG icon repository providing crisp, lightweight

Level, State, and Search Query keywords.

5.3 Page Breakdown & State Flows

1. ‘Dashboard.jsx‘

- **Purpose:** Serves as the central command hub for the student upon logging in.
- **Capabilities:**
- Computes and displays real-time **Profile Strength** (0% to 100%) checking all
- Displays aggregated metric counters: *Matching Scholarships*, *Saved Bookmarks*,
- Alerts student to urgent upcoming application deadlines (within 7, 14, or 30 days).
- Quick action shortcuts to trigger live discovery or edit profile.

2. ‘Scholarships.jsx‘

- **Purpose:** Primary scholarship discovery and exploration interface.
- **Capabilities:**
- Dual modes: *Curated Scholarships* (curated database records) and *Live AI
- Multi-tier filtering controls: Filter by Minimum Match Score, Category, Education
- Tabbed category switching: *All*, *Eligible Only*, *Needs Verification*, *High Value

(>Rs. 50,000)*.

portals.

- Loading skeleton indicators during live search pipeline execution.

3. ‘Saved.jsx‘ ([Saved.jsx](file:///d:/Scholar%20AI/frontend/src/pages/Saved.jsx))

- **Purpose:** Private student repository of bookmarked scholarship opportunities.
- **Capabilities:**
- Fetches saved records directly from ‘/api/saved-scholarships/‘.
- Enables instant removal (unsave) with optimistic UI updates.
- Quick "Apply Now" launchpad directing students to official external application
- Direct bridge to add any saved scholarship into the Application Tracker.

([Details.jsx](file:///d:/Scholar%20AI/frontend/src/pages/Details.jsx))

4. 'Applications.jsx'

- **Purpose:** Full lifecycle application management pipeline.
- **Capabilities:**
- Five distinct pipeline stages: 'DRAFT', 'APPLIED', 'UNDER_REVIEW', 'ACCEPTED',
- Application editing modal: update submission dates, application reference IDs,
- Instant status transition buttons to advance applications through stages.

5. 'Profile.jsx'

- **Purpose:** Student academic, geographic, and demographic data configuration.
- **Fields Managed:**
- Full Name, Email (read-only/synchronized), Phone Number.
- State of Domicile (Indian States & UTs dropdown).
- Social Category ('GENERAL', 'OBC', 'SC', 'ST', 'EWS').
- Gender ('MALE', 'FEMALE', 'OTHER').
- Education Level ('10TH', '12TH', 'DIPLOMA', 'UNDERGRADUATE', 'POSTGRADUATE',
- Degree Course (B.Tech, MBBS, B.Com, B.Sc, BCA, etc.) and Specialization.
- Annual Family Income (INR numeric ceiling).
- Academic Score (Percentage or CGPA on a 10-point scale).
- Age (in years) and Disability Status (Yes/No with percentage).
- **Validation:** Live input bounds checking (e.g., CGPA ≤ 10, percentage $\le

6. 'Details.jsx'

- **Purpose:** In-depth audit view of a single scholarship's eligibility breakdown.
- **Capabilities:**
- Displays full scheme description, award breakdown, application process, and official
- Visual **Rule-by-Rule Audit Matrix**: Renders green checkmarks for passed rules,

7. 'Auth.jsx' ([Auth.jsx](file:///d:/Scholar%20AI/frontend/src/pages/Auth.jsx))

- **Purpose:** User authentication portal supporting Sign In and Sign Up.
- **Capabilities:**
- Tabbed interface with seamless switching.
- Client-side validation: email format regex, password length (≥ 6 characters),
- Automatic token persistence to 'localStorage' and global 'AuthContext' initialization

Chapter 6: Backend Implementation

6.1 Application Architecture & Lifespan

The backend is built with FastAPI, adhering to a strict **Controller-Service-Repository**

querying Serper, scraping HTML with BeautifulSoup, and sanitizing extracted text.


```
@asynccontextmanager
```

and manage database projections.

6.2 CORS & Security Middleware

‘app/main.py‘ configures ‘CORSMiddleware‘ with explicit origin whitelisting:

- Local development origins: ‘http://localhost:5173‘, ‘http://127.0.0.1:5173‘,
- Deployed production origins: ‘https://scholar-ai.vercel.app‘, Vercel preview
- Allows standard HTTP methods (‘GET‘, ‘POST‘, ‘PUT‘, ‘PATCH‘, ‘DELETE‘, ‘OPTIONS‘)

queries or business algorithms directly; they validate inputs via Pydantic schemas,

(‘search_cache.py‘).

6.3 Layered Architecture Details

```
+-----------------------------------------------------------+
```

algorithmic workflows; computing match percentages, running the 9-point rule engine,

PyMongo calls, map BSON ‘_id‘ to string IDs, handle compound uniqueness constraints,

cryptographic hashing and token signing (‘security.py‘), and two-tier search caching

1. **Routers (‘app/api/‘):** Thin HTTP controllers. They do not execute database

   schemas, and status codes are fully referenced below.
2. **Services (‘app/services/‘):** Stateless business logic handlers. They execute
3. **Repositories (‘app/database/‘):** Pure data access objects. They encapsulate
4. **Core Infrastructure (‘app/core/‘):** Manages configuration (‘config.py‘),

Chapter 7: API Documentation

Scholar AI provides 12 dedicated REST API routers exposing 32 endpoints. Complete details,

7.1 Router Overview Table

Router Prefix	Responsibility	Auth Required	Key Endpoints
‘/api/auth‘	User registration, aut..	No (public)	‘POST /register‘, ‘POS..
‘/api/student‘	Student demographic & ..	Yes (Bearer)	‘GET /profile‘, ‘POST ..
‘/api/scholarships‘	Curated database schol..	Read: No / Write: Yes	‘GET /‘, ‘GET /{id}‘, ..
‘/api/eligibility‘	Standalone eligibility..	Yes (Bearer)	‘POST /evaluate‘, ‘POS..
‘/api/matching‘	Match scoring & rankin..	Yes (Bearer)	‘POST /score‘, ‘POST /..
‘/api/explanation‘	Natural language eligi..	Yes (Bearer)	‘POST /explain‘
‘/api/dashboard‘	Dashboard metrics, sta..	Yes (Bearer)	‘GET /stats‘, ‘GET /de..
‘/api/saved-scholarshi..	Bookmark saving, unsav..	Yes (Bearer)	‘GET /‘, ‘POST /‘, ‘DE..
‘/api/applications‘	Application tracking p..	Yes (Bearer)	‘GET /‘, ‘POST /‘, ‘PA..
‘/api/search‘	Full-text & filtered d..	No (public)	‘GET /‘
‘/api/personalized-sea..	Personalized search qu..	Yes (Bearer)	‘POST /generate-queries‘
‘/api/scholarship-disc..	Live Google Serper sea..	Yes (Bearer)	‘POST /discover‘

*(Refer to [API_DOCUMENTATION.md](file:///d:/Scholar%20AI/docs/API_DOCUMENTATION.md) for

Chapter 8: Database Documentation

The platform uses **MongoDB Atlas** (Cloud Replica Set). PyMongo communicates with the

8.1 Collections & Index Architecture

```
erDiagram
```

authenticated user and their academic profile.

preventing a user from bookmarking the same scholarship twice, guaranteeing

```
string provider
```

application status.

‘backend/app/core/security.py‘:

documents automatically after 86,400 seconds (24 hours).

[DATABASE_DOCUMENTATION.md](file:///d:/Scholar%20AI/docs/DATABASE_DOCUMENTATION.md) for

cryptographic security, Scholar AI implements a dual-hasher security architecture in

8.2 Compound & Performance Indexes

1. ‘users.email‘: Unique index ensuring no duplicate accounts exist.
2. ‘students.user_id‘: Unique index enforcing a strict 1-to-1 relationship between an
3. ‘saved_scholarships.(user_id, scholarship_id)‘: **Compound Unique Index**
4. ‘applications.(user_id, scholarship_id)‘: Fast compound lookup index for checking
5. ‘search_cache.created_at‘: **MongoDB TTL (Time-To-Live) Index** set to expire

*(Refer to

Chapter 9: Authentication and Security

9.1 Multi-Hasher Architecture & Automatic Upgrades

To resolve password hash compatibility across legacy migrations while upholding top-tier

‘Authorization: Bearer <token>‘ header on all protected API calls.


```
Incoming Password
```

9.2 3-Tier Email Normalization & Lookup

To eliminate user login failures caused by accidental trailing spaces or inconsistent

1. **Tier 1 (Exact Match):** Direct match on the input string.
2. **Tier 2 (Lowercase Normalized):** Matches ‘email.strip().lower()‘.
3. **Tier 3 (Case-Insensitive Regex Anchor):** Queries MongoDB using ‘{"email":

9.3 JWT Token Lifecycle

- **Algorithm:** ‘HS256‘ signed using a cryptographically secure ‘SECRET_KEY‘.
- **Payload Claims:**
- ‘sub‘: User ID string (‘ObjectId‘).
- ‘email‘: User’s primary email.
- ‘iss‘: ‘"scholar-ai"‘.
- ‘exp‘: Unix timestamp representing 7 days from token issuance.
- **Client Handling:** Stored in browser ‘localStorage‘; injected via HTTP

Chapter 10: Scholarship Discovery Pipeline

The scholarship discovery pipeline bridges the real-time public web with student profiles.

Lakh per annum"', and converts them to integer rupees ('250000').

```
Personalized Profile

                    75,000"', '"Rs. 25,000/-"'.

                    eliminating erroneous low-digit false positives (fixing legacy bugs where "Part 2"

                    9 distinct criteria. Rather than using an unexplainable "black-box" machine learning model,
```

10.1 Rupee Award Amount Extraction Heuristic

Extracting financial award amounts from arbitrary web text requires differentiating between

'scholarship_extraction_service.py' implements:

1. **Lakh & Crore Multipliers:** Captures expressions like '"Rs. 2.5 Lakhs"', '"Rs 1.5
2. **Direct Numeric INR Extraction:** Captures formats like '"Rs. 50,000"', '"INR
3. **The Rs. 500 Minimum Floor Guard:** Rejects extracted numbers below Rs. 500,

Chapter 11: Eligibility Rule Engine

Scholar AI's eligibility engine ('backend/app/services/eligibility_engine.py') evaluates

11.1 The 9 Evaluation Rules

#	Rule Name	Evaluation Logic	Result States
1	**Income Rule**	Verifies 'student.fami..	'PASSED', 'FAILED', 'N..
2	**Category Rule**	Compares 'student.cate..	'PASSED', 'FAILED', 'N..
3	**State Domicile Rule**	Verifies student's sta..	'PASSED', 'FAILED', 'N..
4	**Education Level Rule**	Validates current educ..	'PASSED', 'FAILED', 'N..
5	**Course Synonym Rule**	Matches degree courses..	'PASSED', 'FAILED', 'N..
6	**Score Cutoff Rule**	Converts CGPA to perce..	'PASSED', 'FAILED', 'N..
7	**Age Limit Rule**	Verifies 'student.age ..	'PASSED', 'FAILED', 'N..
8	**Gender Rule**	Enforces gender criter..	'PASSED', 'FAILED', 'N..
9	**Disability Rule**	If scholarship specifi..	'PASSED', 'FAILED', 'N..

ordering.

11.2 Status Classification Matrix

- **‘ELIGIBLE‘:** Every specified criteria evaluated resulted in ‘PASSED‘. The student
- **‘NEEDS_VERIFICATION‘:** No criteria failed, but one or more rules could not be

PASSED
score cutoff requirement.

- **‘INELIGIBLE‘:** At least one mandatory criterion resulted in ‘FAILED‘ (e.g.,

Chapter 12: Scholarship Matching and Scoring

The match score is a normalized mathematical representation of how closely a student

12.1 Mathematical Formulation

$$\text{Evaluated Rules} = \text{Passed Rules} + \text{Failed Rules}$$

$$\text{Match Percentage} = \begin{cases}

100.0, & \text{if Evaluated Rules} = 0 \\

\operatorname{round}\left(\left(\frac{\text{Passed Rules}}{\text{Evaluated

\end{cases}$$

toast.

12.2 Multi-Tier Sorting Hierarchy

Results are sorted using a strict tuple-based ranking hierarchy:

1. **Status Priority (Descending):**
   The student sees that they satisfied 3 of 4 criteria, with a clear alert explaining the 75%
   - ‘ELIGIBLE‘ $\to$ Rank 1
   - ‘NEEDS_VERIFICATION‘ $\to$ Rank 2
   - ‘INELIGIBLE‘ $\to$ Rank 3
2. **Match Score (Descending):** Highest percentage match appears first.
3. **Alphabetical Title (Ascending):** Secondary tie-breaker for consistent UI
   request is sent in the background. If the request fails, the UI reverts with an error

12.3 Worked Scoring Example

A student profile is evaluated against a Corporate Engineering Scholarship:

- **Criteria 1 (Income):** Student: Rs. 2.4 Lakhs, Limit: Rs. 5.0 Lakhs $\to$

reference.

- **Criteria 2 (Category):** Scholarship: Open to all $\to$ **PASSED**
- **Criteria 3 (Course):** Student: B.Tech, Required: Engineering $\to$ **PASSED**
- **Criteria 4 (Score):** Student: 68%, Cut-off: 75% $\to$ **FAILED**
- **Criteria 5 (Age):** Unspecified in listing $\to$ **NOT APPLICABLE**
  the UI updates instantly (icon fills, counter increments). An asynchronous HTTP

$$\text{Passed} = 3, \quad \text{Failed} = 1, \quad \text{Evaluated} = 4$$

$$\text{Match Score} = \left(\frac{3}{4}\right) \times 100 = 75.00\%$$

Result: Status: ‘INELIGIBLE‘ (due to score cutoff failure), Match Score: 75.00%.

Chapter 13: Saved Scholarships Feature

13.1 Architecture & Idempotency

The Saved Scholarships feature allows students to bookmark opportunities for future

- **Optimistic UI:** When the user clicks the bookmark icon on ‘ScholarshipCard.jsx‘,

- **Database Idempotency:** The database enforces a **compound unique index** on

- **Privacy Isolation:** Every query explicitly scopes searches by the authenticated

creates user record.

Chapter 14: Application Tracker Feature

14.1 Application State Machine

The Application Tracker allows students to organize their scholarship applications across

```
[ DRAFT ]          [ APPLIED ]          [ UNDER_REVIEW ]
```

14.2 Lifecycle Transitions

1. **'DRAFT':** Student has initiated preparation, gathered documents, but has not yet
2. **'APPLIED':** Student submitted the application. The system records the
3. **'UNDER_REVIEW':** Application is currently being verified by government or trust
4. **'ACCEPTED':** Application successfully approved for funding.
5. **'REJECTED':** Application rejected (with optional reason notes for future
6. **'AWARDED':** Financial scholarship grant disbursed to student's bank account.

Chapter 15: Complete End-to-End Workflows

Workflow 1: New Student Registration and Profile Creation

1. Student navigates to '/auth' and enters Full Name, Email, and Password.
2. Frontend sends 'POST /api/auth/register'. Backend hashes password with Argon2id and
3. Backend returns JWT token; frontend persists it to 'localStorage' and redirects to
4. Student enters demographic and academic details (12 fields) and clicks "Save
5. Frontend sends 'POST /api/student/profile' with Bearer token.
6. Backend persists student document; Dashboard profile score becomes 100%.

Workflow 2: Live AI Scholarship Discovery

1. Student clicks "Discover Scholarships" on '/scholarships'.
2. Frontend dispatches 'POST /api/scholarship-discovery/discover'.
3. Backend synthesizes search queries based on student's education, course, state, and
4. Backend checks L1/L2 cache. On miss, it queries Serper API, scrubs commercial
5. Rupee amounts and deadlines are extracted; 9 eligibility rules are evaluated per
6. Match scores are computed, results sorted by status and score, and returned to
7. Cards render with match percentages, eligibility badges, and deadline indicators.

Workflow 3: Saving and Bookmarking an Opportunity

1. Student clicks the bookmark icon on a displayed scholarship card.

2. Card optimistically fills the bookmark icon.
3. Frontend sends ‘POST /api/saved-scholarships/‘.
4. Backend executes idempotent upsert in MongoDB Atlas.
5. Scholarship appears on ‘/saved‘ page.

Workflow 4: Tracking an Application to Awarded Status

1. From ‘/saved‘ or ‘/scholarships‘, student clicks "Track Application".
2. Application record is created with status ‘DRAFT‘.
3. Student navigates to ‘/applications‘, enters application reference ID, and sets

4. When funding is disbursed, student advances status to ‘AWARDED‘.
5. Dashboard metrics update to reflect total funds awarded.

Chapter 16: Installation and Local Setup

16.1 Prerequisites

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Python:** Version 3.11 or higher installed and added to PATH.
- **Node.js:** Version 18.0 or higher with ‘npm‘.
- **MongoDB:** MongoDB Atlas free M0 cluster URI or local MongoDB Community Server.
- **Git:** Version 2.30+.

16.2 Backend Setup (Windows 11 PowerShell)

```
# 1. Navigate to backend directory
```

16.3 Frontend Setup (Windows 11 PowerShell)

```
# 1. Open a new PowerShell terminal and navigate to frontend directory
```

Open browser to ‘http://localhost:5173‘.

Chapter 17: Environment Variables Reference

17.1 Backend Environment Variables (‘backend/.env‘)

clustering, CGPA $\times 9.5 \leftrightarrow \%$ conversion, boundary conditions.

junk domain blacklist, deadline extraction, rejection of $< Rs. 500$ false positives.

Variable Name	Required	Default Value	Description
‘PROJECT_NAME‘	No	'"Scholar AI"‘	Application name displ..
‘ENVIRONMENT‘	No	'"development"‘	'"development"', '"sta..
‘MONGODB_URL‘	**Yes**	None	MongoDB Atlas connecti..
‘DATABASE_NAME‘	No	'"scholar_ai"‘	Target database name i..
‘SECRET_KEY‘	**Yes**	None	64+ char random hexade..
‘ALGORITHM‘	No	'"HS256"‘	JWT token signature al..
‘ACCESS_TOKEN_EXPIRE_M..	No	‘10080‘ (7 days)	JWT session validity d..
‘SERPER_API_KEY‘	No	'""‘ (mock fallback)	API key from serper.de..
‘CORS_ORIGINS‘	No	‘http://localhost:5173..	Comma-separated list o..

SHA-256 cache key generation, circuit-breaker fallback on DB outage.

lifecycle, compound unique index deduplication, per-user privacy isolation.

machine (DRAFT $\to$ APPLIED $\to$ UNDER_REVIEW $\to$ ACCEPTED/REJECTED $\to$

17.2 Frontend Environment Variables (‘frontend/.env‘)

Variable Name	Required	Default Value	Description
‘VITE_API_URL‘	No	'""‘ (relative ‘/api‘)	Production backend URL..

Chapter 18: Testing and Quality Assurance

The Scholar AI backend features an automated test suite containing **53 passing unit and

```
============================== 53 passed in 4.82s ==============================
```

18.1 Test Module Directory

- ‘test_auth_security.py‘ & ‘test_auth_login_resilience.py‘: Multi-hasher verification
- ‘test_eligibility_engine_enhancements.py‘: 9-rule evaluation, course synonym
- ‘test_scholarship_extraction.py‘: Serper snippet parsing, Lakh/Crore amount parser,
- ‘test_search_cache.py‘: L1 in-memory LRU + L2 MongoDB TTL index, deterministic
- ‘test_saved_scholarships_api.py‘ & ‘test_saved_repository_duplicates.py‘: Save/unsave
- ‘test_application_service.py‘ & ‘test_dashboard_deadlines.py‘: Application state

'ENVIRONMENT=production'.

Chapter 19: Deployment Guide

19.1 Backend Deployment on Render

1. Connect your GitHub repository to Render as a **Web Service**.
2. **Environment:** Python 3.
3. **Root Directory:** ‘backend‘.
4. **Build Command:** ‘pip install -r requirements.txt‘.
5. **Start Command:** ‘uvicorn app.main:app --host 0.0.0.0 --port $PORT‘.
6. Add environment variables: ‘MONGODB_URL‘, ‘SECRET_KEY‘, ‘SERPER_API_KEY‘,

19.2 Frontend Deployment on Vercel

1. Import repository into Vercel as a Vite project.
2. **Root Directory:** ‘frontend‘.
3. **Build Command:** ‘npm run build‘.
4. **Output Directory:** ‘dist‘.
5. Set ‘VITE_API_URL‘ to your live Render backend URL.

6. Configure ‘vercel.json‘ rewrites to ensure client-side routing fallback to

```
{
```

non-deterministic outputs, and high API latency. Financial eligibility requires 100%

Chapter 20: Troubleshooting Guide

20.1 Common Errors and Resolutions

Symptom / Error	Root Cause	Verified Resolution
‘ssl.SSLCertVerificationError:..	Windows root CA certificate bu..	Pass ‘tlsCAFile=certifi.where..
‘pwdlib.exceptions.UnknownHash..	User database record contains ..	Implement dual-hasher ‘Passwor..
Login fails for matching email..	Email casing or trailing space..	Apply ‘.strip().lower()‘ i..
Profile strength displays 55% ..	Dashboard checking legacy fiel..	Update ‘calculateProfileScore‘..
Extracted scholarship shows aw..	Extractor matched stray digit ..	Added Rs. 500 minimum floor ru..
Duplicate bottom bar on deskto..	Mobile navigation bar renderin..	Removed duplicate ‘<nav class=..

lightning-fast Rollup bundling.

Chapter 21: Technical Decisions and Tradeoffs

1. **FastAPI vs. Django / Flask:**
- *Decision:* FastAPI.
- *Tradeoff:* Django provides built-in admin and ORM, but is monolithic and synchronous

student profiles.

to government portal biometric and OTP constraints.

2. **MongoDB Atlas vs. PostgreSQL:**
- *Decision:* MongoDB Atlas.
- *Tradeoff:* Relational SQL databases enforce rigid tables, but web-scraped

require manual deadline verification.

3. **Rule Engine vs. Machine Learning / LLM Classification:**
- *Decision:* Deterministic rule engine.
- *Tradeoff:* While LLMs handle ambiguity, they suffer from hallucinations,

caste certificates, and marksheets using Tesseract / Vision AI to auto-populate

4. **Vite vs. Create-React-App:**
- *Decision:* Vite 5.
- *Tradeoff:* CRA is deprecated and slow. Vite provides sub-second HMR and

for approaching scholarship deadlines.

Chapter 22: Limitations and Future Scope

22.1 Limitations

- **External Portal Application:** The platform cannot submit applications directly due

- **Scraper Heuristics:** Highly obfuscated JavaScript-rendered portals occasionally

for API-based scholarship verification.

22.2 Future Roadmap

1. **Document Vault & AI Verification:** OCR-based scanning of income certificates,

2. **WhatsApp & SMS Push Alerts:** Automated Twilio / WhatsApp Business notifications

3. **Direct Integration with State Portals:** Partnering with state welfare departments

process memory, L2 in MongoDB with TTL auto-expiration.

runtime schemas.

4. **Multilingual Interface:** Support for Hindi, Marathi, Tamil, Telugu, Bengali, and

Chapter 23: Complete Technical Interview Preparation

*(Refer to

results and snippets.

Chapter 24: Technical Glossary

1. **Argon2id:** State-of-the-art hybrid password hashing algorithm offering peak after a specified duration.
2. **Bcrypt:** Adaptive cryptographic hash function based on the Blowfish cipher;
3. **BSON:** Binary JSON serialization format used by MongoDB to store documents and module replacement.
4. **Compound Unique Index:** A multi-field database index enforcing uniqueness across
5. **CORS (Cross-Origin Resource Sharing):** Security mechanism using HTTP headers to
6. **Deterministic Rule Engine:** An evaluation system whose output is solely determined
7. **FastAPI:** High-performance, async-native Python web framework based on Starlette
8. **Glassmorphism:** UI design trend featuring translucent frosted-glass card surfaces
9. **JWT (JSON Web Token):** Compact, URL-safe token format representing
10. **L1 / L2 Cache:** Multi-level caching architecture; L1 resides in fast Python
11. **Pydantic:** Data validation library using Python type annotations to enforce
12. **PyMongo:** Official synchronous Python driver for MongoDB.
13. **Serper API:** Fast Google Search engine API returning structured JSON search
14. **TTL Index (Time-To-Live):** MongoDB index that automatically deletes documents
15. **Vite:** Rapid frontend build tool leveraging native ES modules for instant hot